React Hooks Series #01

useState vs useReducer

What they do, how to write them, and **when to pick which** — explained simply, with examples.

Ali Moradi · linkedin.com/in/ali-moradi-dev

Swipe →

FIRST THINGS FIRST

What is state?

State is a component's **memory**. When it changes, React **re-renders** the component with the new value.

- Typing in an input → the input value
- Clicking "Add to cart" → the cart items
- Opening a modal → isOpen

> React gives you two hooks to manage it: `useState` and `useReducer`.

useState · SYNTAX

One line to add state

```
const [state, setState] = useState<Type>(initialState);
```

`state`	The current value for this render.
`setState`	Updates the value **and** triggers a re-render.
`initialState`	The value on the **first render** only. Ignored after that.
`<Type>`	Optional — usually **inferred**. Add it when it can't be: `useState<User | null>(null)`

useState · EXAMPLE

A simple counter

Counter.tsx

```tsx
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0); // number
  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

Click → `setCount` stores the new number → React re-renders → the button shows it.

useState · 3 TIPS FROM THE DOCS

Use it the right way

1 New value depends on the old one? Pass a function

```
setCount(c => c + 1); // ✅ uses the latest value
```

2 Objects & arrays: replace, don't mutate

```
setUser({ ...user, name: 'Sara' }); // ✅ new object
user.name = 'Sara';                 // ❌ no re-render
```

3 Expensive initial value? Pass the function itself

```
useState<Todo[]>(createTodos);   // ✅ runs once
useState<Todo[]>(createTodos()); // ❌ every render
```

useReducer · SYNTAX

All update logic in one place

```
const [state, dispatch] = useReducer(
  reducer, initialArg, init?
);
```

`reducer`	A pure function: `(state, action) => newState`
`initialArg`	The starting state.
`init?`	Optional. Builds the initial state from `initialArg`.
`dispatch`	Send an **action** → React updates state and re-renders.

> TypeScript **infers** the types from your reducer.

useReducer · STEP 1

Write the reducer

reducer.ts

```ts
type State = { count: number };

type Action =
  | { type: 'incremented' }
  | { type: 'decremented' }
  | { type: 'reset' };

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case 'incremented':
      return { count: state.count + 1 };
    case 'decremented':
      return { count: state.count - 1 };
    case 'reset':
      return { count: 0 };
  }
}
```

> An **action** describes what happened. The **reducer** decides how state changes.

useReducer · STEP 2

Use it in the component

Counter.tsx

```tsx
function Counter() {
  const [state, dispatch] =
    useReducer(reducer, { count: 0 });

  const add = () => dispatch({ type: 'incremented' });
  const reset = () => dispatch({ type: 'reset' });

  return (
    <>
      <p>Count: {state.count}</p>
      <button onClick={add}>+</button>
      <button onClick={reset}>Reset</button>
    </>
  );
}
```

> Typo like `{ type: 'incremnted' }`? TypeScript shows an **error** before you even run it.

useReducer · RULES

Keep your reducer clean

- **Pure:** same input → same output. No fetch, timers or other side effects.
- **Never mutate** state. Always return a new object or array.
- In Strict Mode, React calls it **twice** in development to catch mistakes.

```
// ❌ mutation
state.count++;
return state;

// ✅ new object
return { ...state, count: state.count + 1 };
```

COMMON GOTCHA (BOTH HOOKS)

State is a snapshot

```
function handleClick() {
  setCount(count + 1); // or dispatch(...)
  console.log(count);  // 😮 still the OLD value
}
```

Setting state does **not** change the variable right away. The new value arrives on the **next render**.

> Need the next value now? Calculate it yourself:
> `const next = count + 1;`

SIDE BY SIDE

The differences

	`useState`	`useReducer`
Update with	`setState(value)`	`dispatch(action)`
Logic lives	Spread across event handlers	In **one** reducer function
Code size	Less code up front	More code up front
Readability	Great for simple updates	Better for complex updates
Debugging	Harder to see what changed	Log every action in one place
Testing	Tied to the component	Pure function → test alone

DECISION TIME

When to use which?

`useState`

- Simple, independent values
- Booleans, strings, numbers
- Few ways to update
- e.g. `isOpen`, input text

`useReducer`

- Many values change together
- Many handlers update the same state
- Next state depends on complex logic
- e.g. todo list, multi-step form

Start with useState. Switch to useReducer when updates get hard to follow.

You can also **mix both** in the same component.

RECAP

TL;DR

- `useState` → simple values, quick to write.
- `useReducer` → complex logic, one place, easy to test.
- Both: never mutate, and state updates on the **next render**.

NEXT IN THE SERIES

#02 — useEffect: syncing with the outside world

👍 Like 🔁 Repost ➕ Follow for #02